Design and Analysis of Algorithm

Module VI

Let me know, if any typos and other mistakes in the shared ppt

Module 6

Module No. 6	Branch and Bound	8 Hours
15-puzzle problem, Assignment problem, Knapsack Problem, **Job Sequencing**, Traveling Salesman Problem. **NP-completeness**: Reduction amongst problems, classes NP, NP-complete, and polynomial time reductions.		

- B&B is one of the problem-solving technique used and it is similar to the backtracking.
- It also uses the **state space tree**.
- It is used for solving the maximization problems and minimization problems.
- If a maximization problem given then we can convert it minimization using the Branch and bound technique.

It follows two mechanisms:

- A mechanism to generate branches when searching the solution space

- A mechanism to generate a bound so that many braches can be terminated

Branch-and-bound strategy

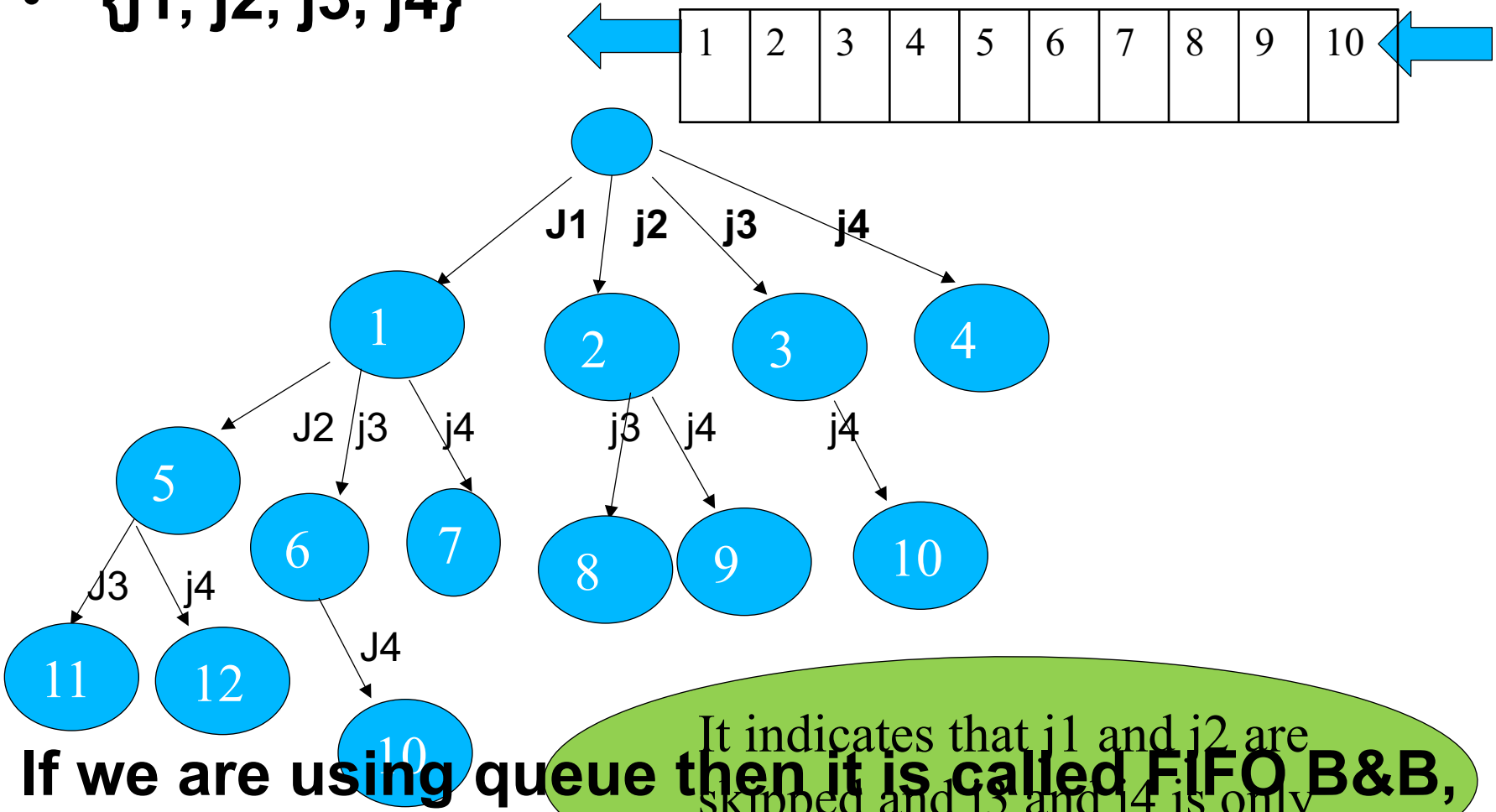
- It is efficient in the average case because many branches can be terminated very early.
- Although it is usually very efficient, a very large tree may be generated in the worst case.
- Many NP-hard problem can be solved by B&B efficiently in the average case; however, the worst case time complexity is still exponential.

Backtracking Vs B&B

- Backtracking ← uses DFS
- B&B ← uses BFS

B&B using Queue →FIFO

- {j1, j2, j3, j4}

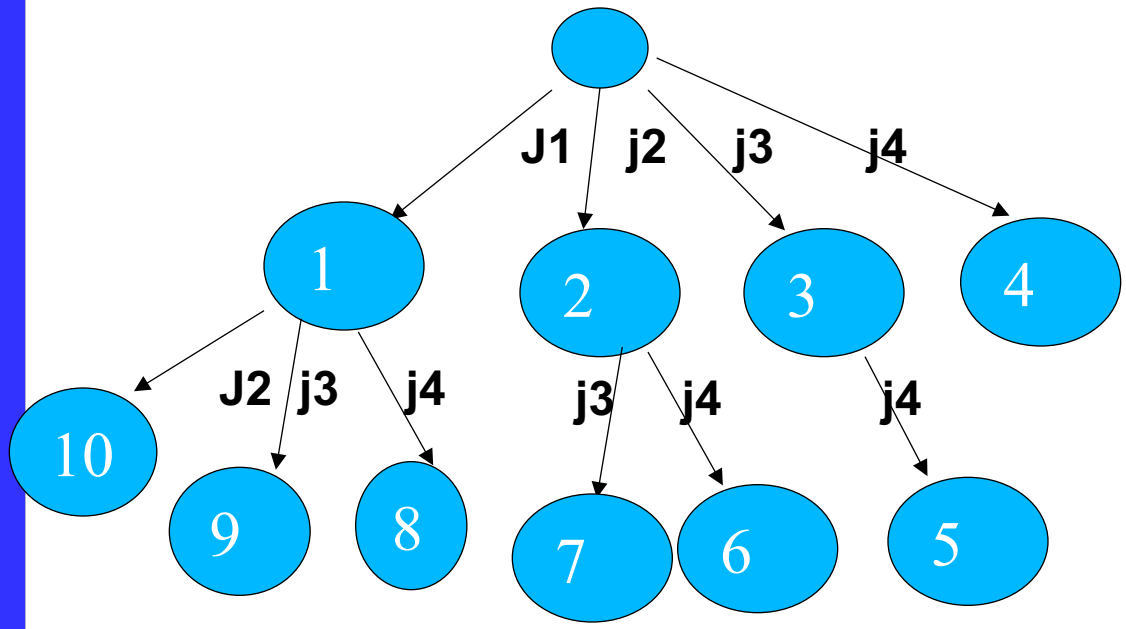


If we are using queue then it is called FIFO B&B, but we are following BFS only

It indicates that j1 and j2 are skipped and j3 and j4 is only considered

B&B using Stack→LIFO

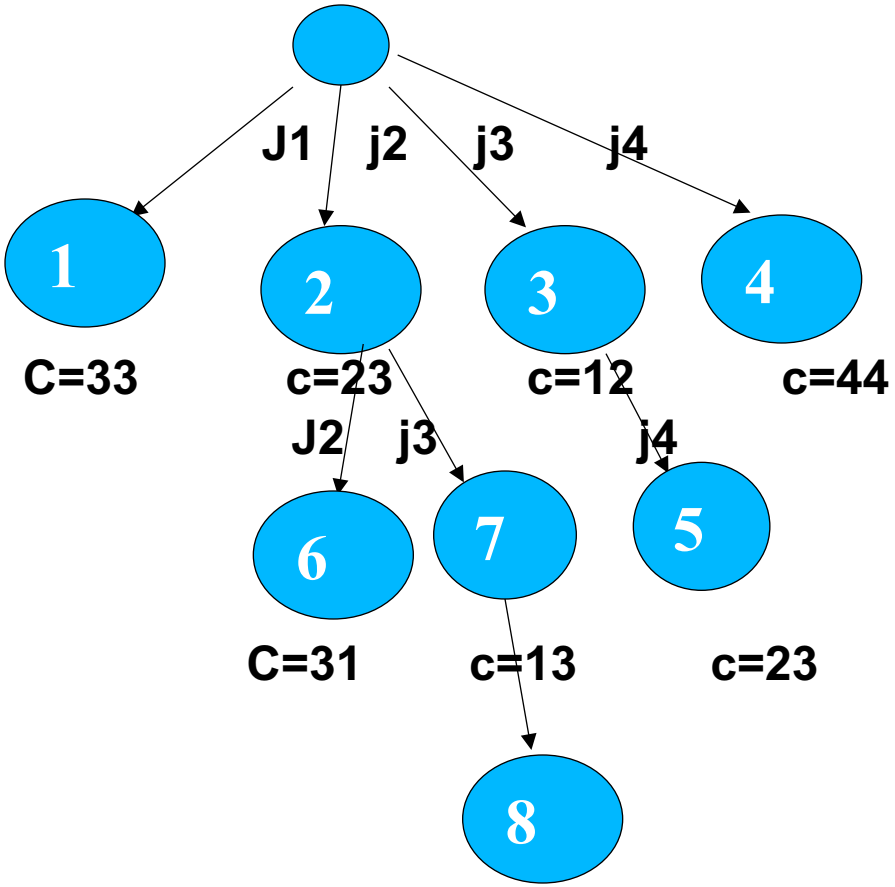
- {j1, j2, j3, j4}



If we are using stack then it is called LIFO B&B, but we are following BFS only

Least Cost (LC) B&B Method

- {j1, j2, j3, j4}



Job Sequencing using B & B

- Job sequencing is a scheduling technique used to maximize profit while adhering to deadlines.

	j1	j2	j3	j4
Profit	5	10	6	3
Deadline	1	2	3	1
P Time	1	2	1	1

- But B&B is used for minimization problems only, so we need to convert maximization problem to minimization problem.
- Profit is converted into penalty

	j1	j2	j3	j4
Penalty	5	10	6	3
Deadline	1	2	3	1
P Time	1	2	1	1

Job Sequencing

- A set of jobs are given
- Each job has a set of defined deadlines and some profit associated with it.
- A job is profited only if that job is completed within the given deadline.
- Another point to note is that only one processor will be available for processing all the jobs.
- The processor will take one unit of time in order to complete a job.

Process

1. S is the set of jobs considered {j1, j2, j3, j4}

2. **C** is the cost i.e sum of penalty till the last job considered {skipped jobs}

3. **U** is the upper bound i,e sum of penalty of jobs not considered in the solution

4. Explore nodes in BFS order

5. Initialize UV(sum of penalty of jobs not considered in the solution) =INF

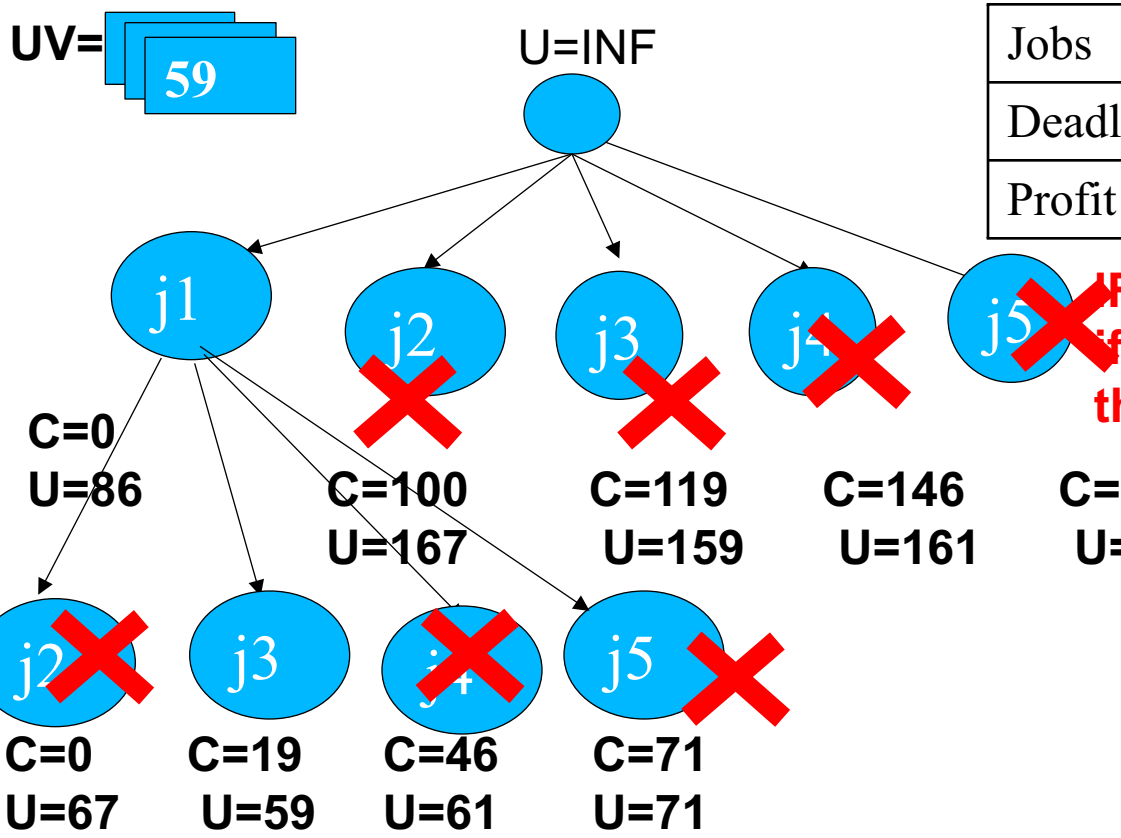
6. For each and every node find cost **C, If C<=UV, then only calculate U value for that node, if U is less than UV then replace with it.**

7. **IF C > UV then kill the branch**, because we will not get the solutions.

Example 1

S is the set of jobs considered {j1, j2, j3, j4}
C is the cost i.e sum of penalty till the last job considered {skipped jobs}
U is the upper bound i,e sum of penalty of jobs not considered in the solution

Jobs	j1	j2	j3	j4	j5
Deadline	2	1	2	1	3
Profit	100	19	27	25	15

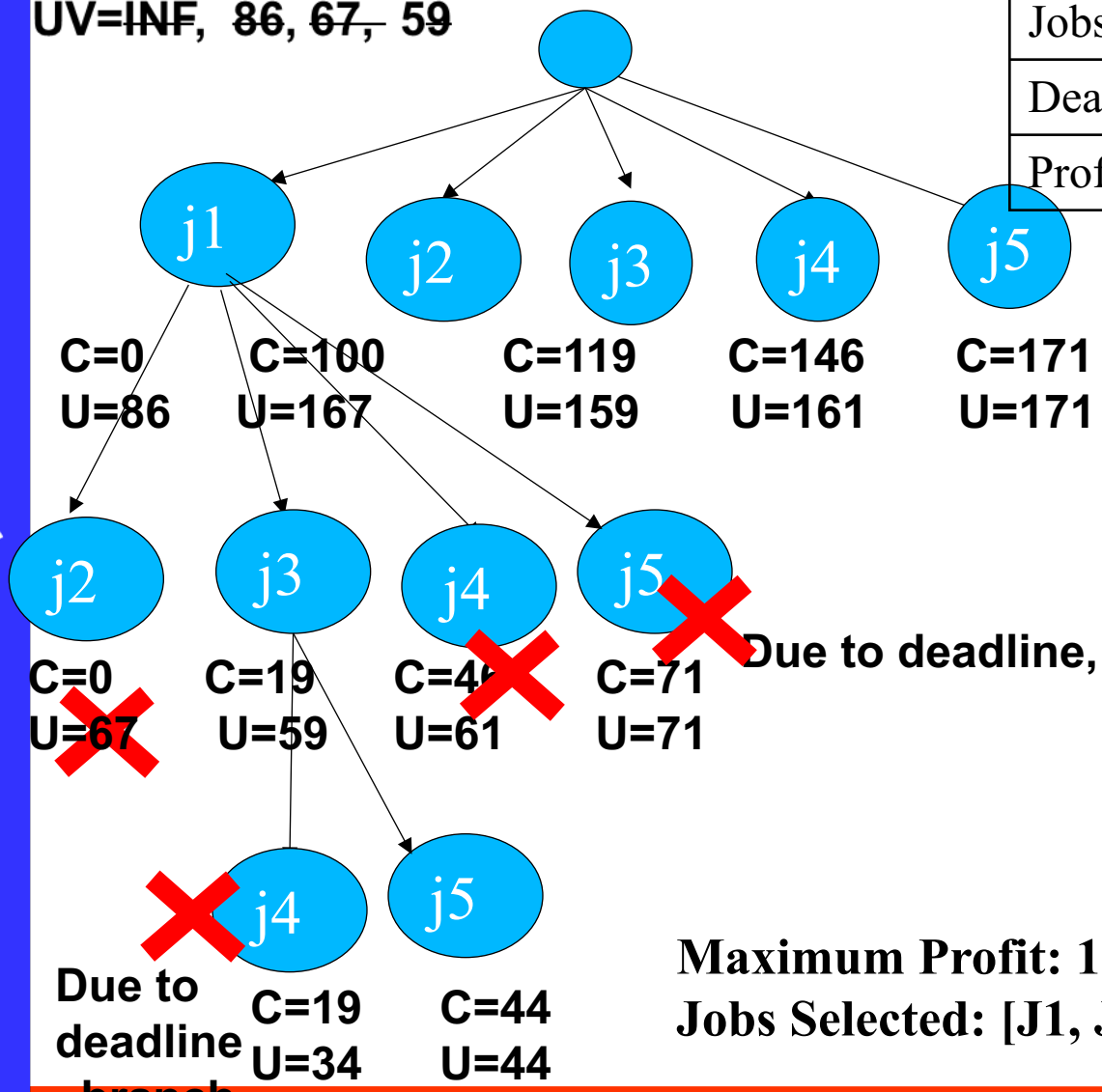


IF C>UV then kill the branch.
if newly calculated U is less than UV then replace it with UV

Here j2 deadline is an issue

Example 1

Jobs	j1	j2	j3	j4	j5
Deadline	2	1	2	1	3
Profit	100	19	27	25	15



IF C>UV then kill the branch.
if newly calculated U is less than UV then replace it with U

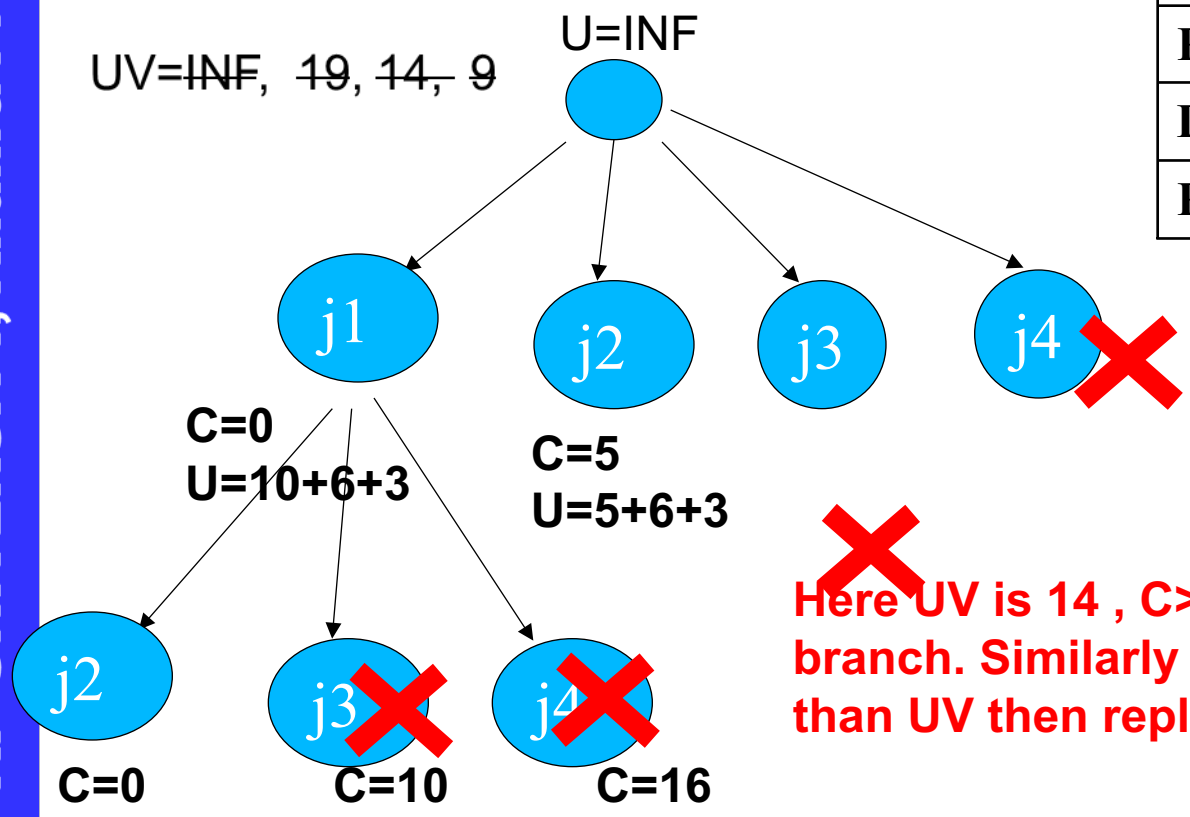
Maximum Profit: 100+27+15=142

Jobs Selected: [J1, J3, J5]

Job Sequencing : Example 2

S is the set of jobs considered {j1, j2, j3, j4}
C is the cost i.e sum of penalty till the last job considered {skipped jobs}
U is the upper bound i,e sum of penalty of jobs not considered in the solution

	j1	j2	j3	j4
Penalty	5	10	6	3
Deadline	1	2	3	1
P Time	1	2	1	1

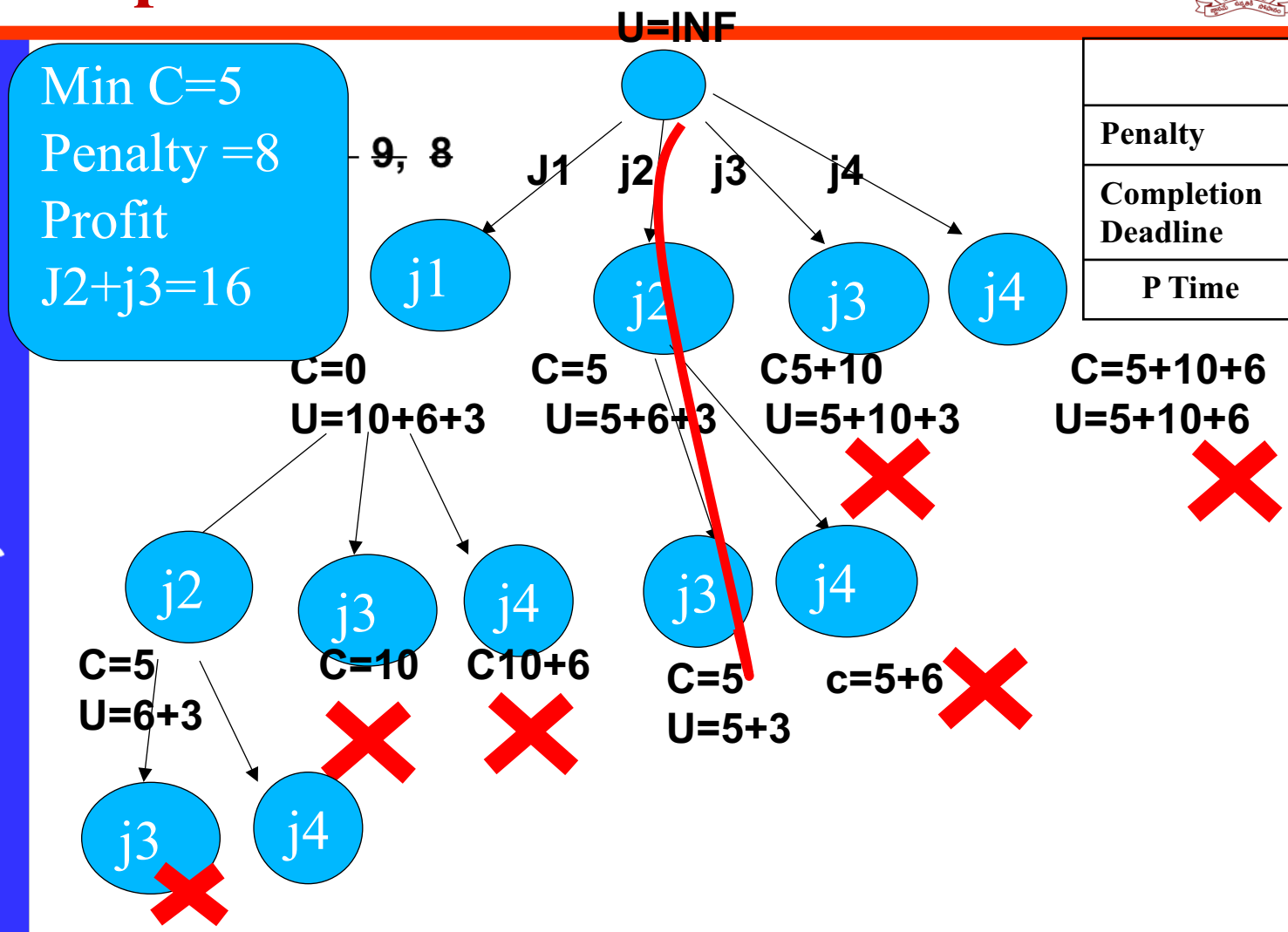


Here UV is 14 , C>UV i.e 15>14, then kill the branch. Similarly if newly calculated U is less than UV then replace it with UV

Here cost is more then the uv value i.e 10 or 16 >9 then kill the branch

Example 2

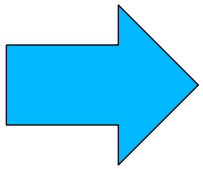
	j1	j2	j3	j4
Penalty	5	10	6	3
Completion Deadline	1	2	3	1
P Time	1	2	1	1



It can't be extended bcz deadline, Time for (j1+j2) is (1+2=3) which > j3 deadline

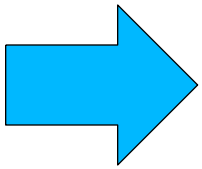
Examples

Job('J1', 1, 35),
Job('J2', 2, 9),
Job('J3', 3, 27),
Job('J4', 4, 25),
Job('J5', 5, 15)



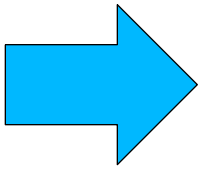
Maximum Profit: 102
Jobs Selected: ['J1', 'J3', 'J4', 'J5']

Job('J1', 1, 35),
Job('J2', 2, 9),
Job('J3', 3, 27),
Job('J4', 4, 25),
Job('J5', 5, 35)



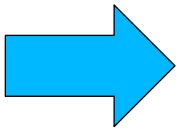
Maximum Profit: 122
Jobs Selected: ['J1', 'J5', 'J3', 'J4']

Job('J1', 1, 35),
Job('J2', 2, 49),
Job('J3', 1, 27),
Job('J4', 2, 25),
Job('J5', 3, 35)



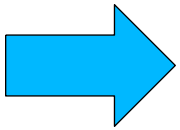
Maximum Profit: 84
Jobs Selected: ['J2', 'J5']

Job('J1', 2, 35),
Job('J2', 3, 29),
Job('J3', 1, 27),
Job('J4', 2, 25),
Job('J5', 1, 37)



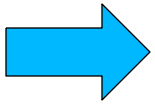
Maximum Profit: 101
Jobs Selected: ['J5', 'J1', 'J2']

Job('J1', 3, 35),
Job('J2', 2, 29),
Job('J3', 1, 27),
Job('J4', 2, 25),
Job('J5', 1, 37)



Maximum Profit: 72
Jobs Selected: ['J5', 'J1']

Job('J1', 3, 35),
Job('J2', 2, 29),
Job('J3', 1, 47),
Job('J4', 2, 25),
Job('J5', 3, 37)



Maximum Profit: 119
Jobs Selected: ['J3', 'J5', 'J1']

Assignment Problem Using Branch & Bound

The assignment problem: Given **n** persons, n jobs and the effectiveness of each person for each job, the problem is to assign each person to one and only one job in such a way that the measure of effectiveness is optimized (**Maximized or Minimized**).

Assignment problem

- Example

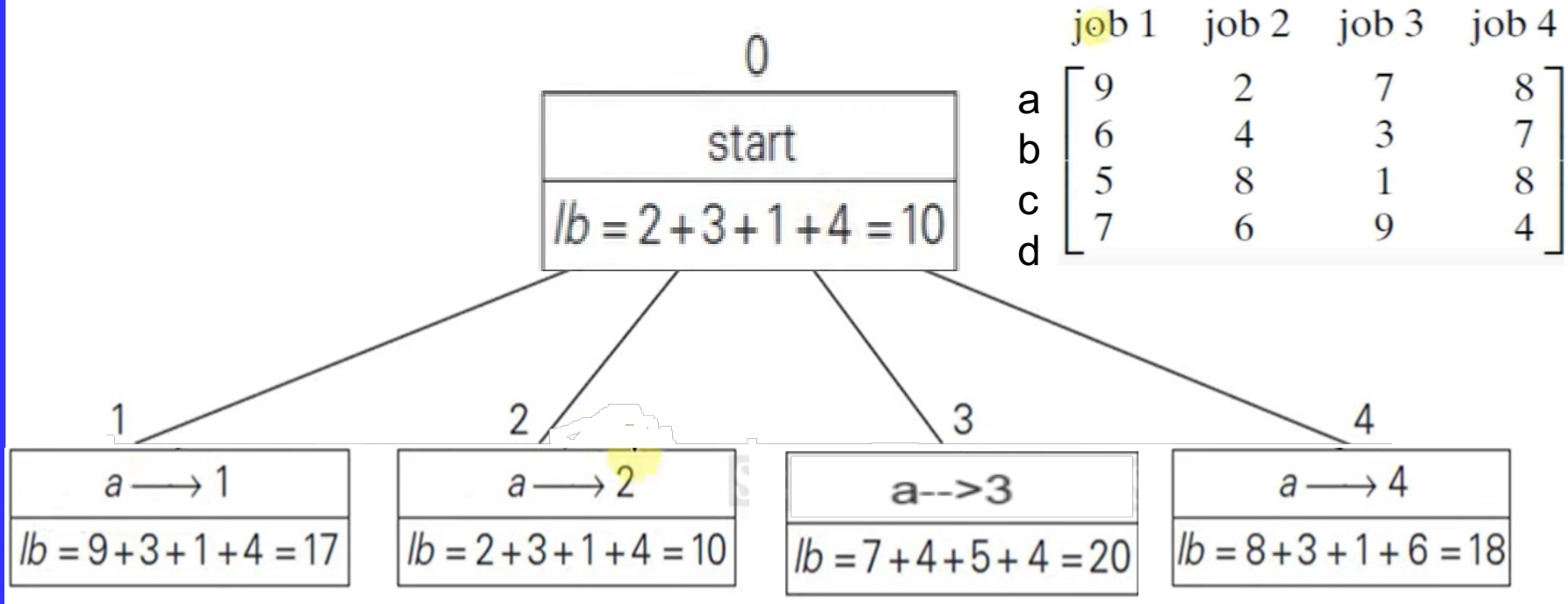
Jobs	j1	j2	j3	J4
Persons	a	b	c	d

$$C = \begin{array}{c} \begin{matrix} J1 & j2 & j3 & j4 \end{matrix} \\ \begin{bmatrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{bmatrix} \end{array} \begin{matrix} \text{person } a \\ \text{person } b \\ \text{person } c \\ \text{person } d \end{matrix}$$

Objective is to minimize the cost.

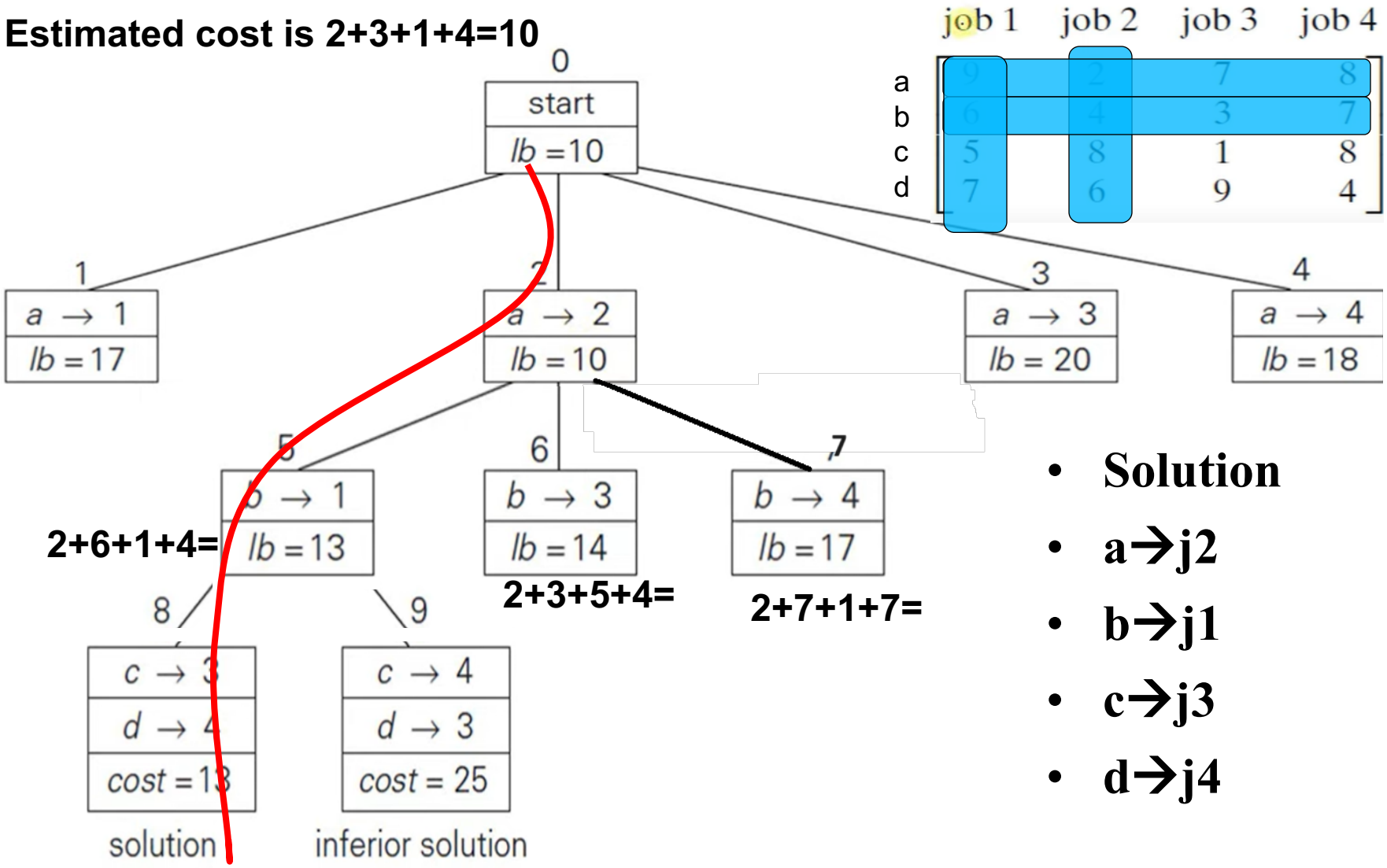
Find a lower bound of the problem (LB).

a→j2, b→j3, c→j3, d→j4, total cost is 2+3+1+4=10



- Start with the estimated lower bound

Estimated cost is 2+3+1+4=10



- **Solution**
- **a→j2**
- **b→j1**
- **c→j3**
- **d→j4**

Assignment problem:Example 3

Minimum Cost of each Row

	Job 1	Job 2	Job 3	Job 4
Person 1 (A)	11	4	9	10
Person 2 (B)	8	6	5	9
Person 3 (C)	7	10	3	10
Person 4 (D)	9	8	11	6

So that Lower Bound (LB) = 4+5+3+6 = 18

Possibility 1: Person 1 (A) does Job 1

➔ Person 1 (A) does Job 1

	Job 1	Job 2	Job 3	Job 4
Person 1 (A)	11	4	9	10
Person 2 (B)	8	6	5	9
Person 3 (C)	7	10	3	10
Person 4 (D)	9	8	11	6

Then LB for probability 1 is : 11+5+3+6 = 25

Assignment problem

➔ Person 1 (A) does Job 2

	Job 1	Job 2	Job 3	Job 4
Person 1 (A)	11	4	9	10
Person 2 (B)	8	6	5	9
Person 3 (C)	7	10	3	10
Person 4 (D)	9	8	11	6

Then LB for probability 2 is : 4+5+3+6 = 18

Assignment problem

➔ Person 1 (A) does Job 3

	Job 1	Job 2	Job 3	Job 4
Person 1 (A)	11	4	9	10
Person 2 (B)	8	6	5	9
Person 3 (C)	7	10	3	10
Person 4 (D)	9	8	11	6

Then LB for probability 3 is : 9+6+7+6 = 28

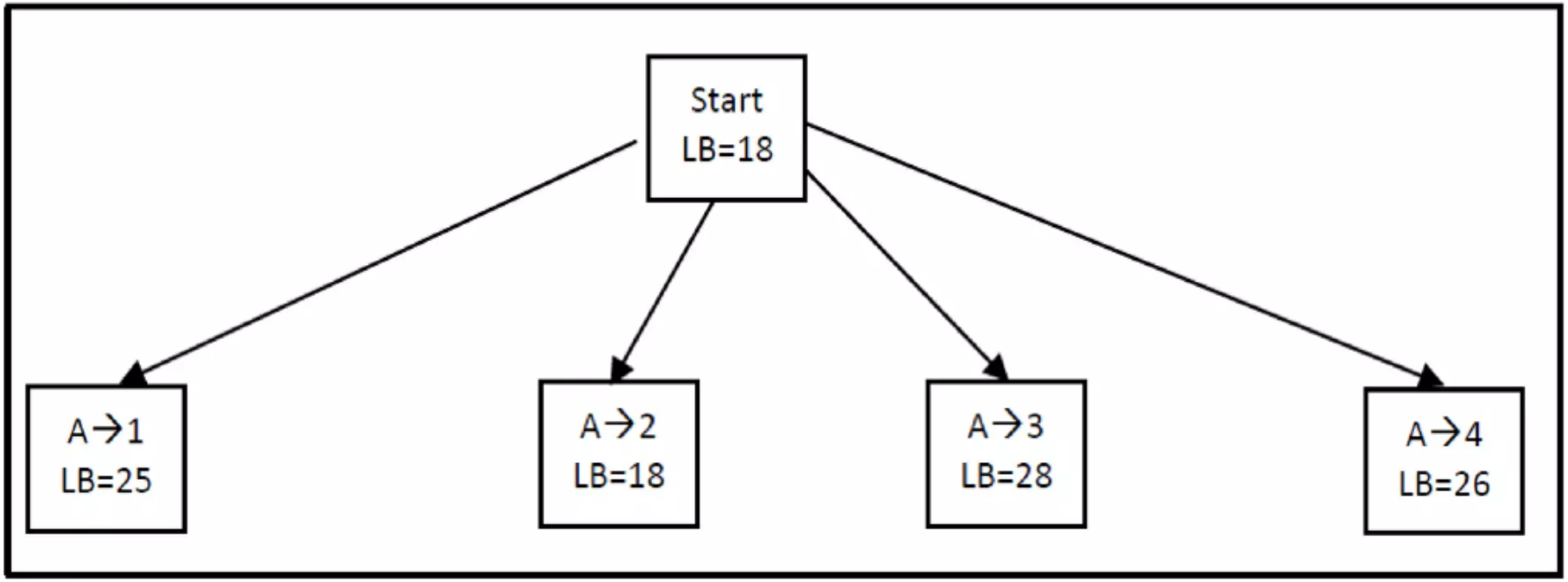
Assignment problem

➔ Person 1 (A) does Job 4

	Job 1	Job 2	Job 3	Job 4
Person 1 (A)	11	4	9	10
Person 2 (B)	8	6	5	9
Person 3 (C)	7	10	3	10
Person 4 (D)	9	8	11	6

Then LB for probability 4 is : 10+5+3+8 = 26

Assignment problem



the node with the minimum value to be expanded is selected, namely A→2 with LB = 18. Then the second person (B) is chosen to do the job or assignment.

Possibility 1: Person 2 (A) does Job 1

➔ Person 2 (B) does Job 1

	Job 1	Job 2	Job 3	Job 4
Person 1 (A)	11	4	9	10
Person 2 (B)	8	6	5	9
Person 3 (C)	7	10	3	10
Person 4 (D)	9	8	11	6

Then LB for probability 1 is :4+8+3+6 = 21

Assignment problem

➔ Person 2 (B) does Job 3

	Job 1	Job 2	Job 3	Job 4
Person 1 (A)	11	4	9	10
Person 2 (B)	8	6	5	9
Person 3 (C)	7	10	3	10
Person 4 (D)	9	8	11	6

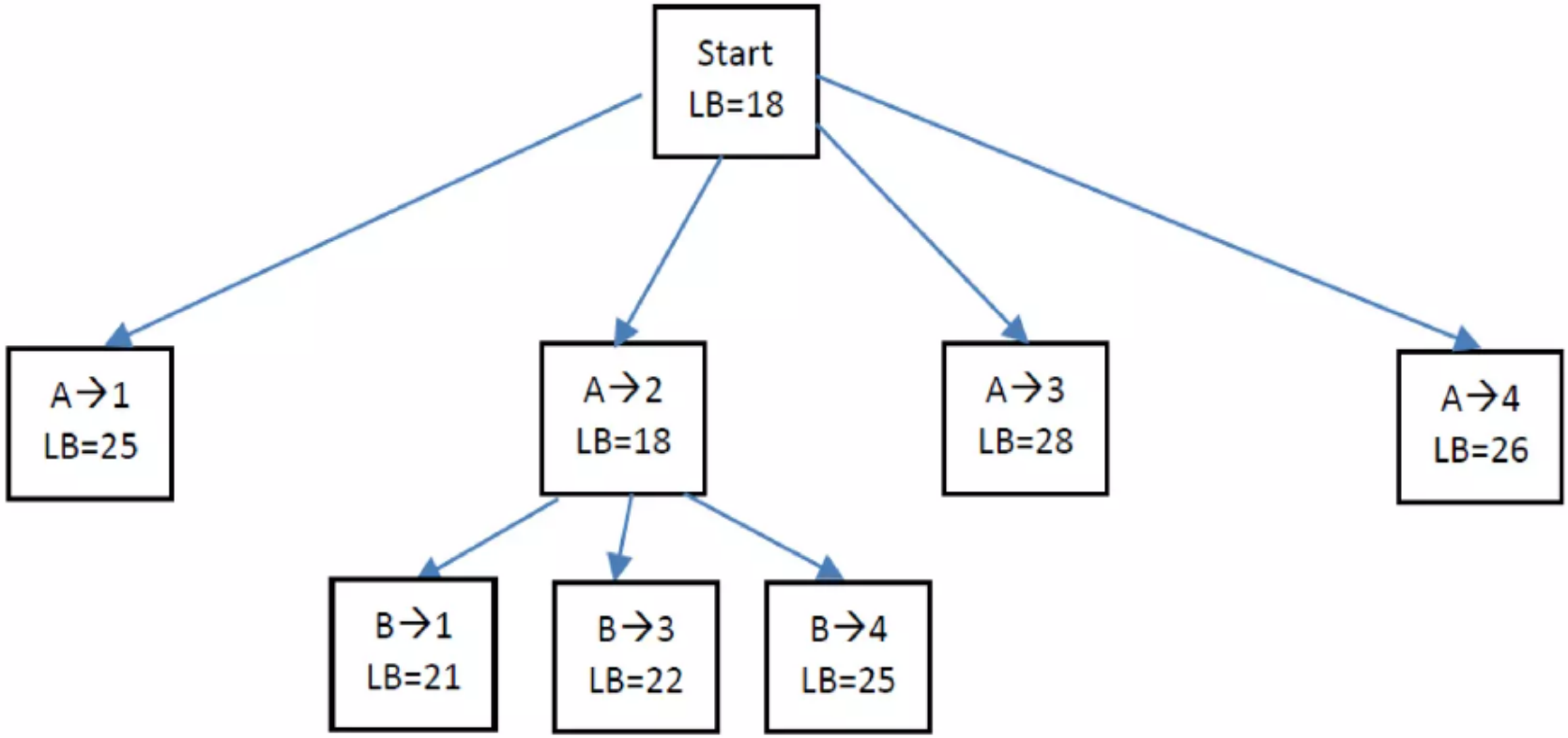
Then LB for probability 2 is :4+5+7+6 = 22

➔ Person 2 (B) does Job 4

	Job 1	Job 2	Job 3	Job 4
Person 1 (A)	11	4	9	10
Person 2 (B)	8	6	5	9
Person 3 (C)	7	10	3	10
Person 4 (D)	9	8	11	6

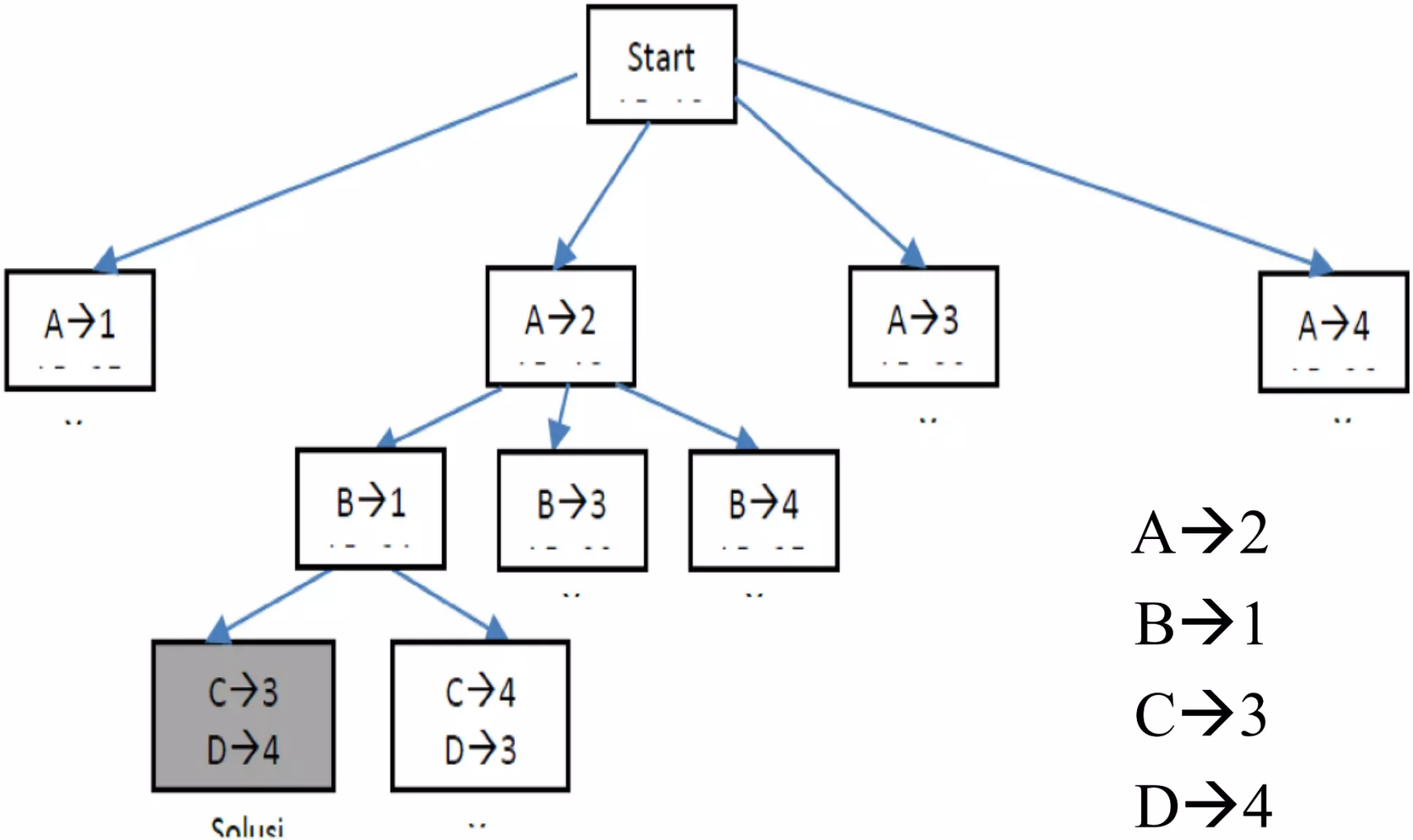
Then LB for probability 3 is :4+9+3+9 = 25

Assignment problem



Next, select the node with the minimum cost to expand, namely B→1

Assignment problem



Example 2

	j1	j2	j3	J4
P1	6	7	3	9
P2	7	4	2	8
P3	6	9	5	2
p4	7	2	5	3

15 Puzzle Problem Using Branch & Bound

Objective

- The problem consist of 15 numbered (0-15) tiles on a square box with 16 tiles (one tile is blank or empty).

- The objective of this problem is to change the arrangement of initial node to goal node by using series of legal moves.

15 Puzzle Problem

The problem consist of 15 numbered (0-15) tiles on a square box with 16 tiles(one tile is blank or empty).

The objective is to place the numbers on tiles to match the final configuration using the empty space.

We can slide four adjacent (**left, right, above, and below**) tiles into the empty space.

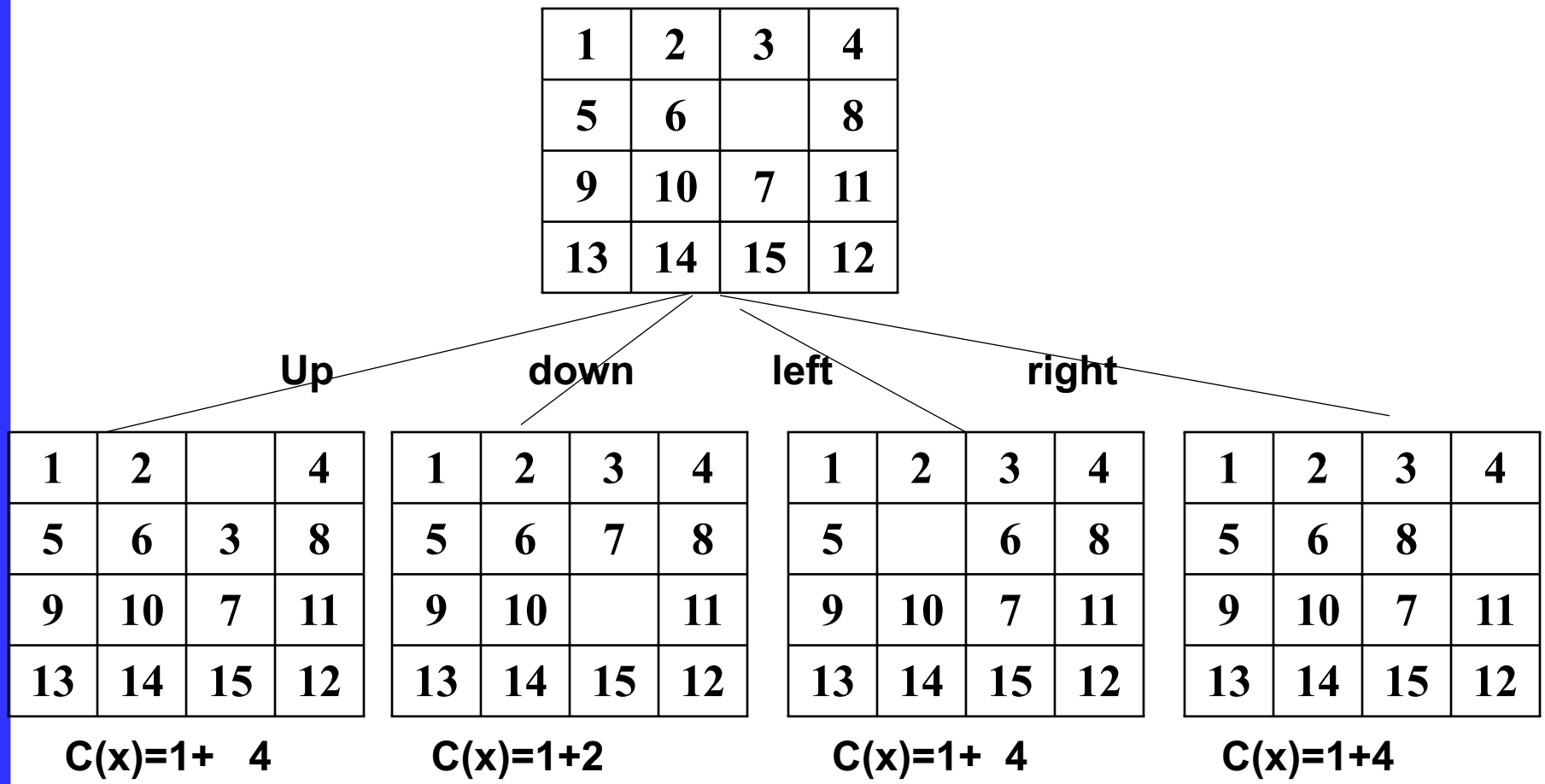
Each and every move creates a new arrangement, and this arrangement is called state of puzzle problem.

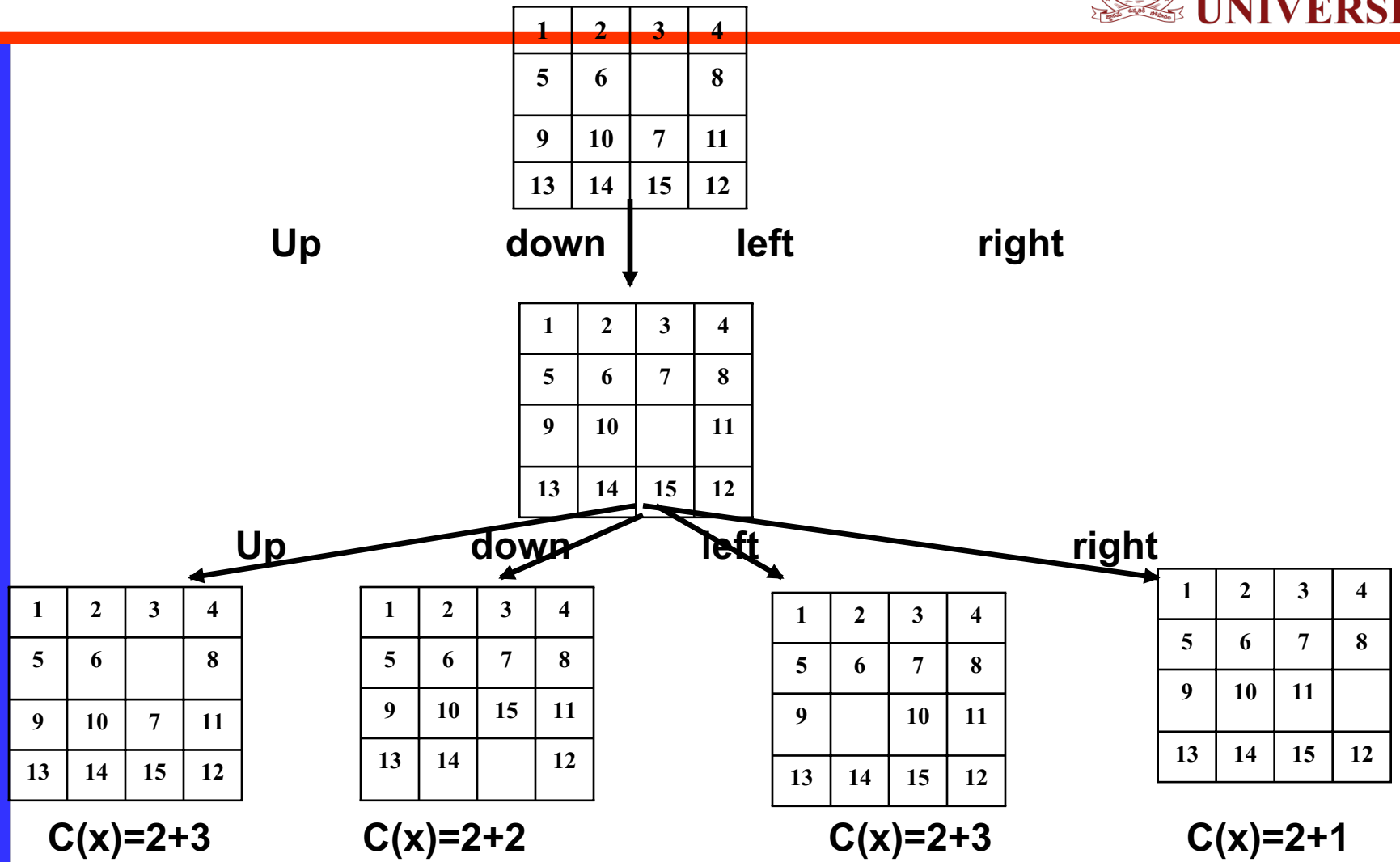
By using different states, a **state space tree diagram** is created, in which edges are labeled according to the direction in which the empty space moves.

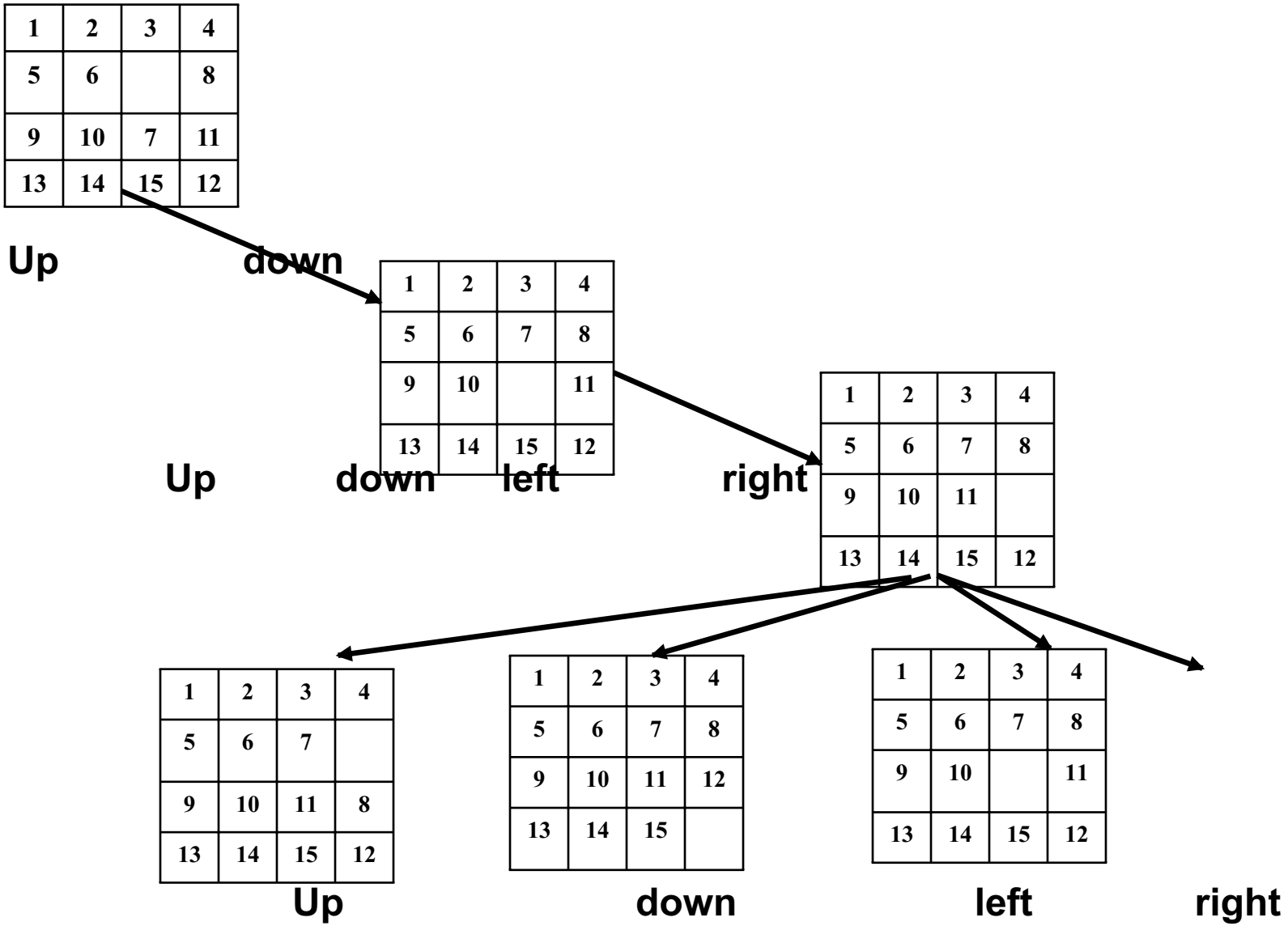
15 Puzzle Problem

- The **state space tree** is very large because it can be 16! different arrangements.
- In **state space tree**, nodes are numbered as per the level.
- In each level we must calculate the value or cost of each node by using given formula:
- **C(x)=f(x)+g(x),**
- **f(x) is length of path from root or initial node to node x, g(x) is estimated length of path from x downward to the goal node. i.e. Number of non blank tile not in their correct position**.
- Each time node with smallest cost is selected for further expansion towards goal node.
- This node become the e-node. State Space tree with node cost is shown in diagram.

15-puzzle problem




1 2 3 4
5 6 8
9 10 7 11
13 14 15 12
Up
down
left
right
1 2 3 4
5 6 7 8
9 10 11
13 14 15 12
Up
down
left
right
1 2 3 4
5 6 8
9 10 7 11
13 14 15 12
C(x)=2+3
1 2 3 4
5 6 7 8
9 10 15 11
13 14 12
C(x)=2+2
1 2 3 4
5 6 7 8
9 10 11
13 14 15 12
C(x)=2+3
1 2 3 4
5 6 7 8
9 10 11
13 14 15 12
C(x)=2+1


1 2 3 4
5 6 8
9 10 7 11
13 14 15 12
Up
down
1 2 3 4
5 6 7 8
9 10 11
13 14 15 12
Up
down
left
right
1 2 3 4
5 6 7 8
9 10 11
13 14 15 12
1 2 3 4
5 6 7
9 10 11 8
13 14 15 12
Up
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15
down
1 2 3 4
5 6 7 8
9 10 11
13 14 15 12
left
right

Knapsack Problem Using Branch & Bound

0/1 Knapsack Problem: LC B&B

- The given problem is maximization problem, but we can't solve it using B&B.
- We need to convert into a minimization problem.
- We need to find upper bound and cost.

Items→i	x1	x2	x3	x4
Profit	10	10	12	18
weight	2	4	6	9
M=15				

$$C = \sum_{i=1}^{n}, (p_i * x_i) \& W = \sum_{i=1}^{n} w_i, with\ fraction$$

$$UB = \sum_{i=1}^{n}, (p_i * x_i), without\ fraction,$$

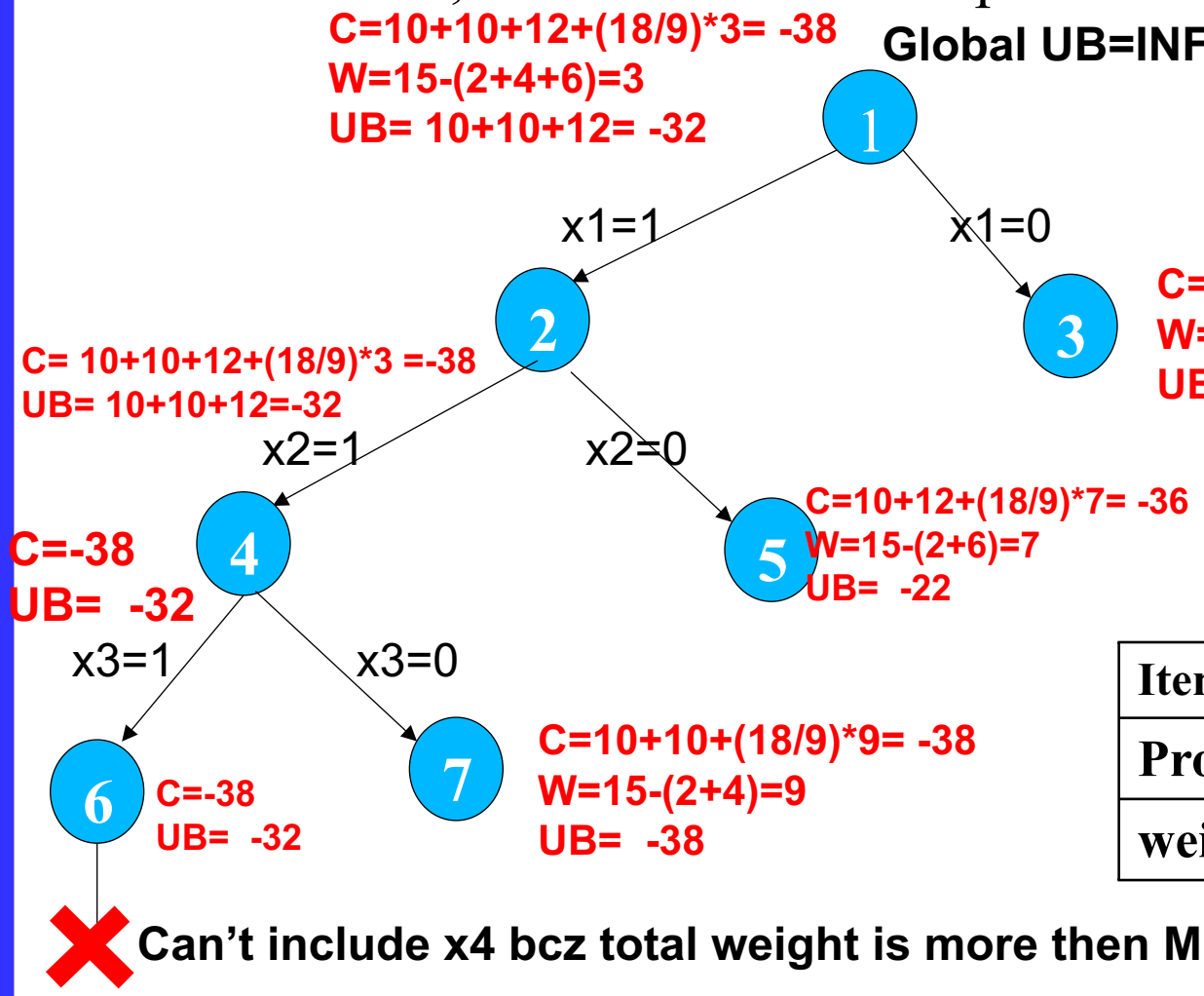
Fraction is used for calculation, but not included in the problems

Procedure

- Let Global UV is INF initially
- Calculate Cost (C) with fractions and calculate UV without fractions
- If any node UV is less then Global UV the update Global UV with node UV.
- If any node UV is greater then Global UV than, do not kill the branch, **but find the least UV to explore next level.**
- If cost C is more then the **GUV than kill the branch.**

Knapsack Problem: LC B&B

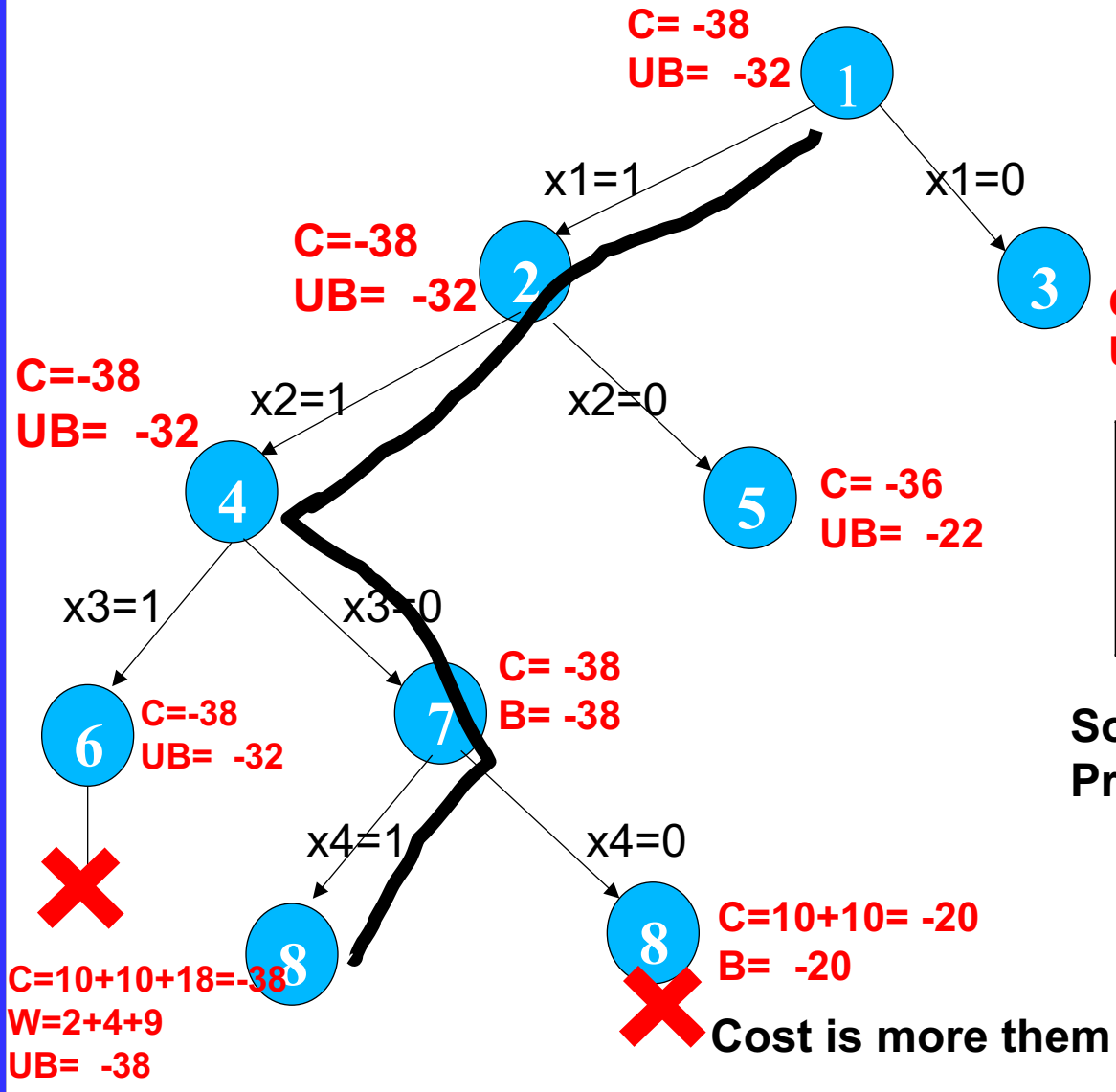
- As the problem is maximization, we need to convert into minimization, so need to consider profit as cost in negative.



Items→	x1	x2	x3	x4
Profit	10	10	12	18
weight	2	4	6	9

Can't include x4 bcz total weight is more then M

Knapsack Problem: LC B&B



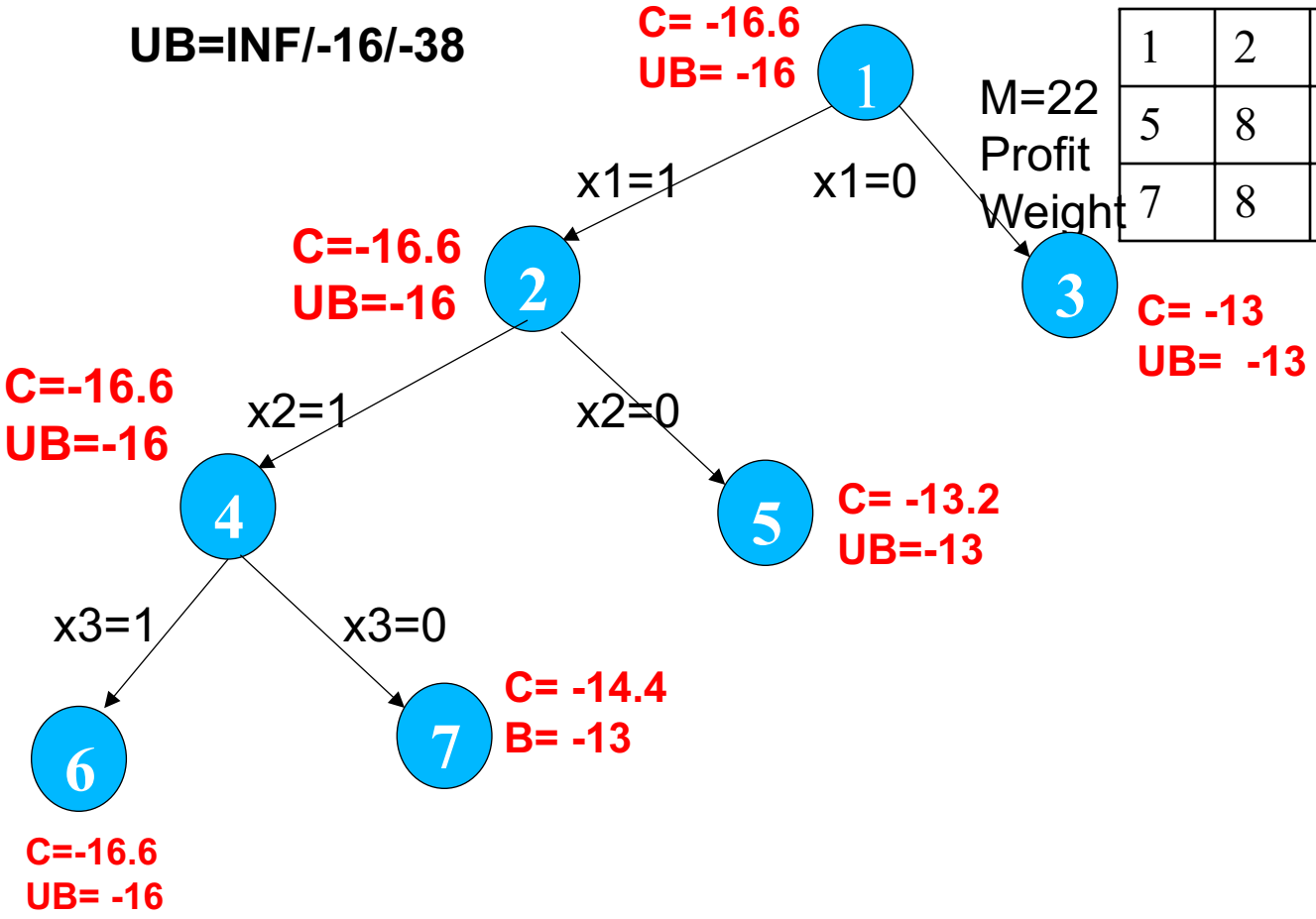
UB=INF/-32/-38

Items→	I1	I2	I3	I4
Profit	10	10	12	18
weight	2	4	6	9

Solution is x={1, 1, 0, 1}
Profit is 10+10+18=38

Cost is more them UB, so terminate the node

Example 2

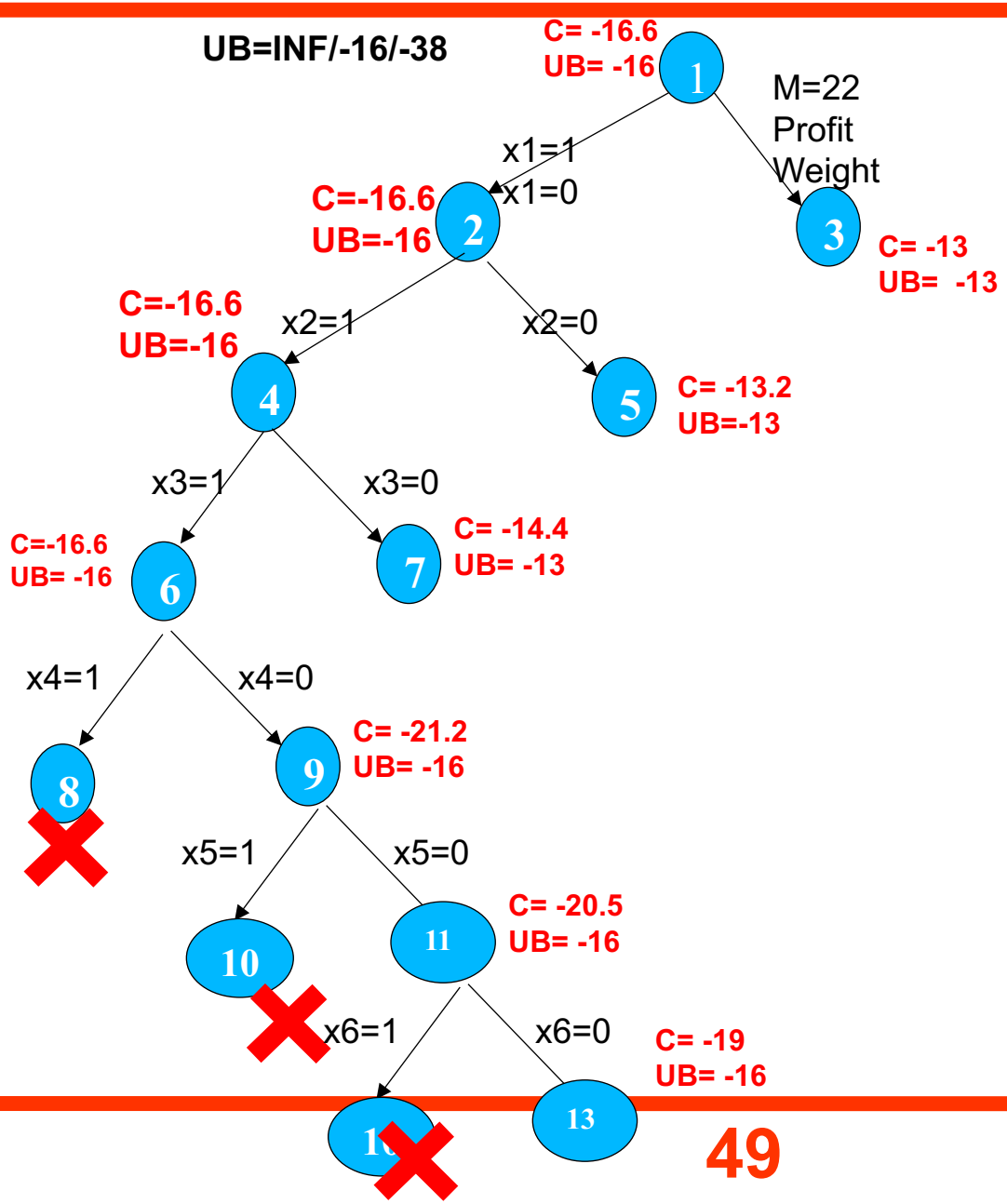


1	2	3	4	5	6	7
5	8	3	2	7	9	4
7	8	4	10	4	6	4

M=22
Profit
Weight

Example 2

1	2	3	4	5	6	7
5	8	3	2	7	9	4
7	8	4	10	4	6	4



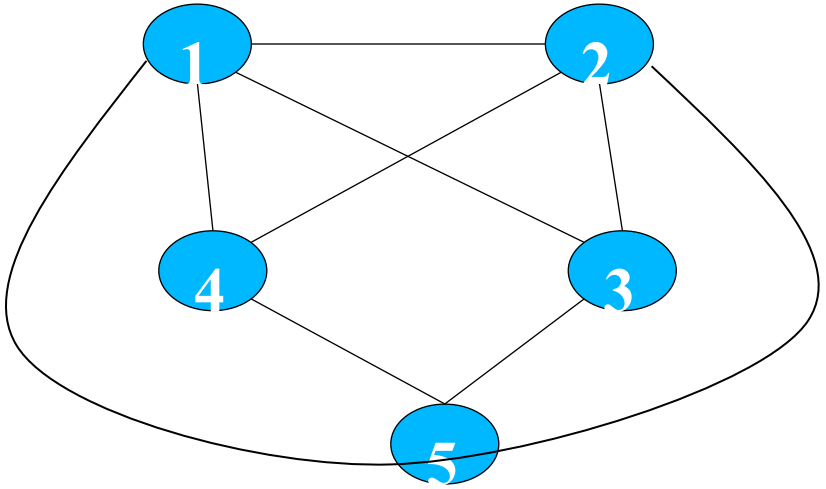
Travelling Salesman Problem

Find the reduced matrix from the original matrix

- Find the row min of each row
- Subtract row min from all elements of that row
- Find the column min of each column
- Subtract column min from all elements of that column.
- Find ‘r’ is the reduced cost, i.e the sum of all row min + column min sum

Step 1: Finding Cost

INF	20	30	10	11	10
15	INF	16	4	2	2
3	5	INF	2	4	2
19	6	18	INF	3	3
16	4	7	16	INF	4



Subtract row min from row elements

INF	10	20	0	1
13	INF	14	2	0
1	3	INF	0	2
16	3	15	INF	0
12	0	3	12	INF

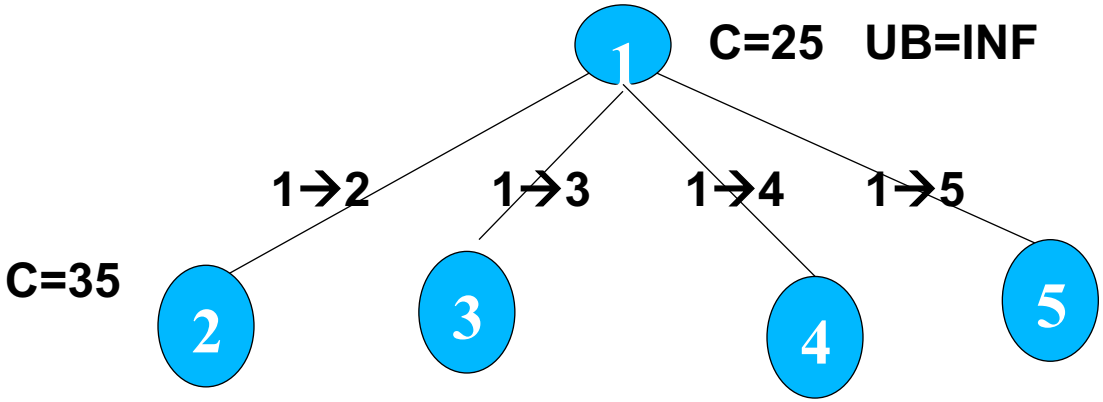
1 0 3 0 0

Subtract col min from col elements

INF	10	17	0	1
12	INF	11	2	0
0	3	INF	0	2
15	3	12	INF	0
11	0	0	12	INF

row min sum+ col min sum **(r)** =25

Generate State space Tree



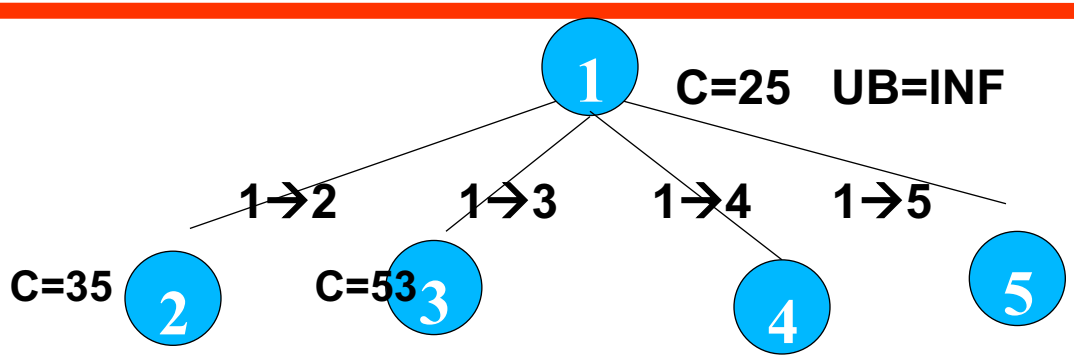
For 1 to 2, we need to calculate cost for that make the 1st row and 2nd column values to INF and as we cant go back to 1 from 2, so make 2→1 as also INF.

INF	INF	INF	INF	INF
INF	INF	11	2	0
0	INF	INF	0	2
15	INF	12	INF	0
11	INF	0	12	INF

C(2)=C(1, 2)+r+r'
r is the reduced cost i.e. 25
r' is the further reduced cost
C(2)=10+25+0=35

Check for all rows having zero or not, if not, take min & subtract from all
Check for all columns having zero nor not, if not, take min & subtract from all

Generate State space Tree



For 1 to 3 we need to calculate cost (3) by making the 1st row and 3rd column values to INF and as we cant go back 3 from 1, so make 3→1 as INF.

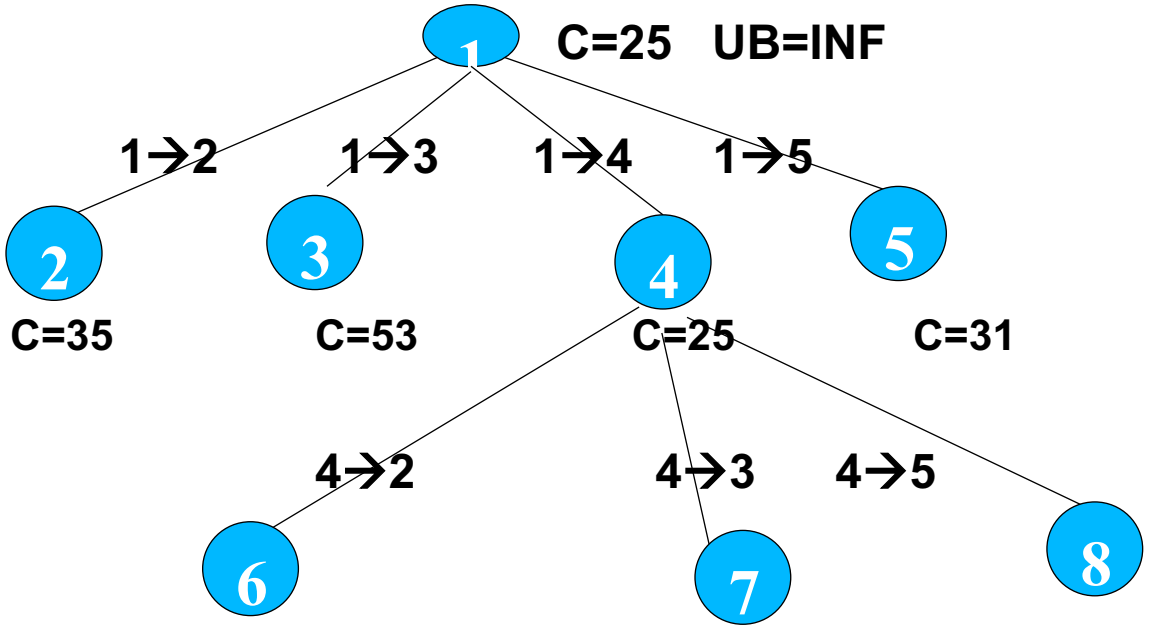
INF	INF	INF	INF	INF
12	INF	INF	2	0
INF	3	INF	0	2
15	3	INF	INF	0
11	0	INF	12	INF
11	0	INF	0	0

INF	INF	INF	INF	INF
1	INF	INF	2	0
INF	3	INF	0	2
4	3	INF	INF	0
0	0	INF	12	INF

C(3)=C(1, 3)+r+r'
C(2)=17+25+11=53

Check for all rows having zero or not, if not, take min & subtract from all
Check for all columns having zero nor not, if not, take min & subtract from all

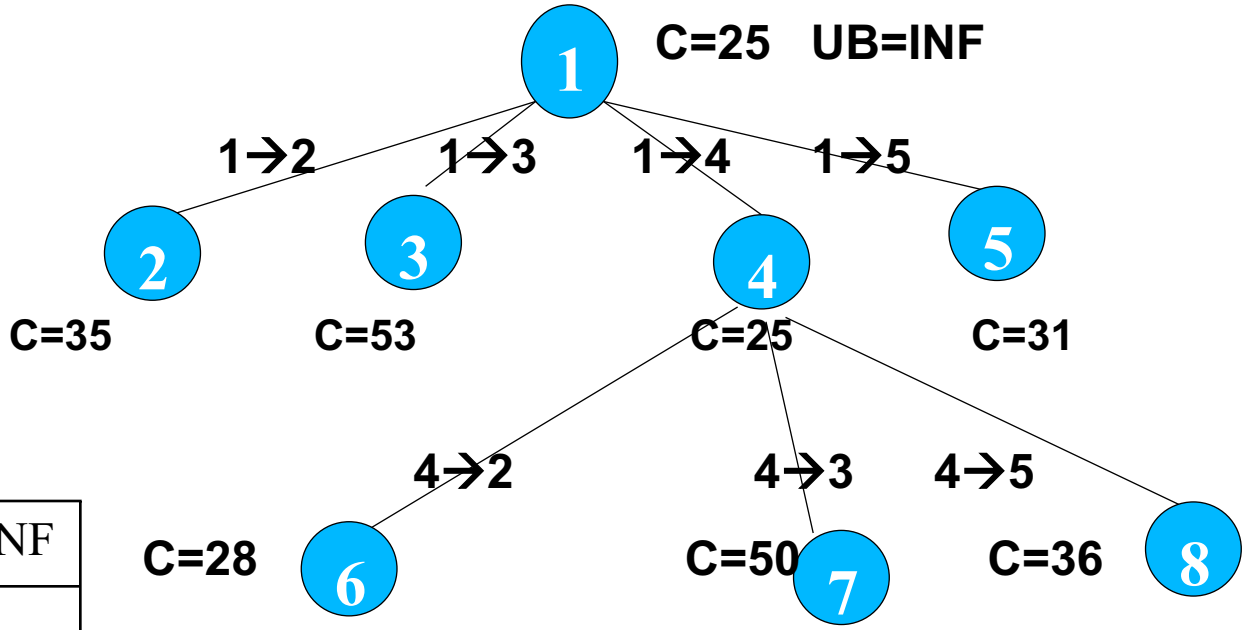
As per B&B bounding condition , we need to expand the least cost one



INF	INF	INF	INF	INF
12	INF	11	INF	0
0	3	INF	INF	2
INF	3	12	INF	0
11	0	0	INF	INF

Consider the matrix that we calculated for node 4

As per B&B bounding condition , we need to expand the least cost one



INF	INF	INF	INF	INF
12	INF	11	INF	0
0	3	INF	INF	2
INF	3	12	INF	0
11	0	0	INF	INF

Consider the matrix that we calculated for node 4

Fro 4→2, we need make 4th row and 2nd column to INF

When we consider 1 to 4 and 4 to 2, we cant go back to 2 to 1, so make (2,1) as INF

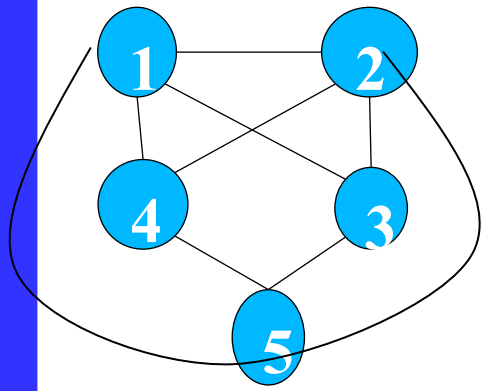
C(6)=C(4,2)+C(4)+r'

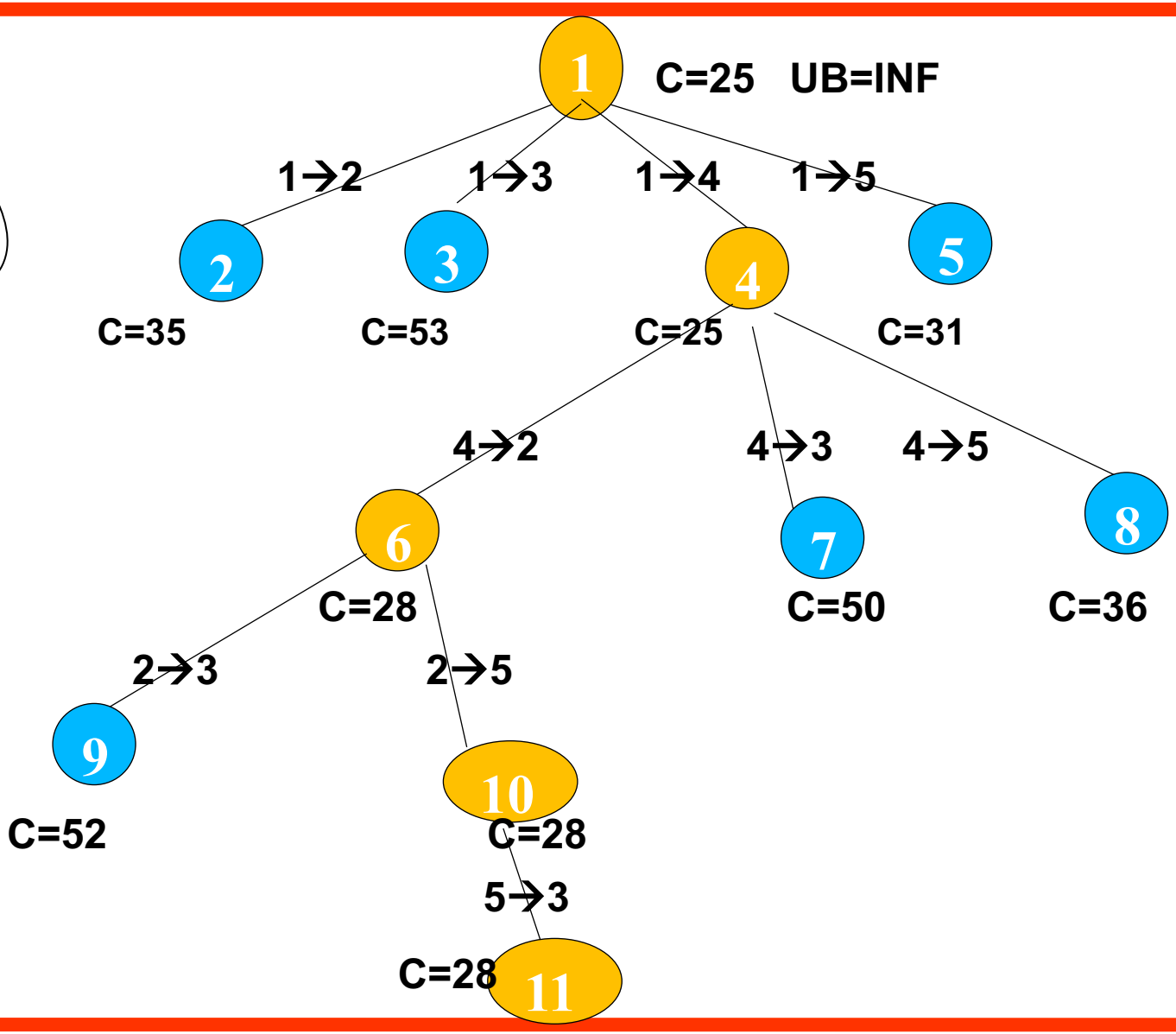
C(6)=3+25+0=28

Check for all rows having zero or not, if not, take min & subtract from all

Check for all columns having zero nor not, if not, take min & subtract from all

Final Step


1
2
4
3
5


1
C=25 UB=INF
1→2
1→3
1→4
1→5
2
3
4
5
C=35
C=53
C=25
C=31
4→2
4→3
4→5
6
7
8
C=28
C=50
C=36
2→3
2→5
9
10
C=52
C=28
5→3
C=28
11

- The worst-case time complexity is **O(n!)**, but in practice, the Branch and Bound method performs significantly better by pruning unnecessary branches. It is particularly useful for small to medium-sized TSP instances where exact solutions are required.

NP Complete
NP Hard

Polynomial Time

Linear Search-O(n)

Binary Search-O(logN)

Insertion Sort-$O(n^2)$

Merge Sort-O(nlogn)

Matrix Multiplication-n^3

Exponential Time

0/1-knapsack-2^n

TSP-2^n

Subset sum-2^n

Graph colouring-2^n

Hamiltonian cycle-2^n

Research objective is to solve the polynomial time algorithms even in less time complexity.
And Exponential time algorithms to solve in polynomial time

Objective

- Our objective is to either to solve exponential time algorithm in polynomial time
- or we need to find a solution for one and find the similarity between them, so that rest of similar problems can be solved later.
- Or just find some similarity between these exponential time algorithms.
- When we can't write deterministic algorithms in polynomial time, then we need to try for nondeterministic algorithms.

Deterministic and Nondeterministic

- Deterministic algorithms are the algorithms for a given particular input, the computer will always produce the same output going through the same states
- but in the case of the non-deterministic algorithm, for the same input, the compiler may produce different output in different runs.
- Its difficult to know how these algorithms are working.

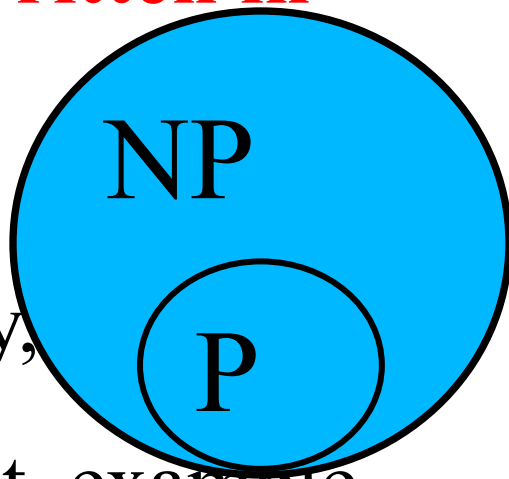
- In a nondeterministic algorithms, most of statements are deterministic, but few statements are nondeterministic, means its difficult to how those statement are working.
- We can preserve that written algorithm, so that nondeterministic statements can be solved by some one in future(**it become research for future**).
-

Example: Nondeterministic algorithm


```
void Search(Arr, N, Key)
{   index=choice();                    1 unit of time
    if(Key==Arr[index])                1 unit of time
    {   write(index);                  1 unit of time
        success();                     1 unit of time
    }
    else
    {   write(0);                      1 unit of time
        failure();                     1 unit of time
    }                                  ==========
}                                      Total   O(c)
```

P and NP Problems

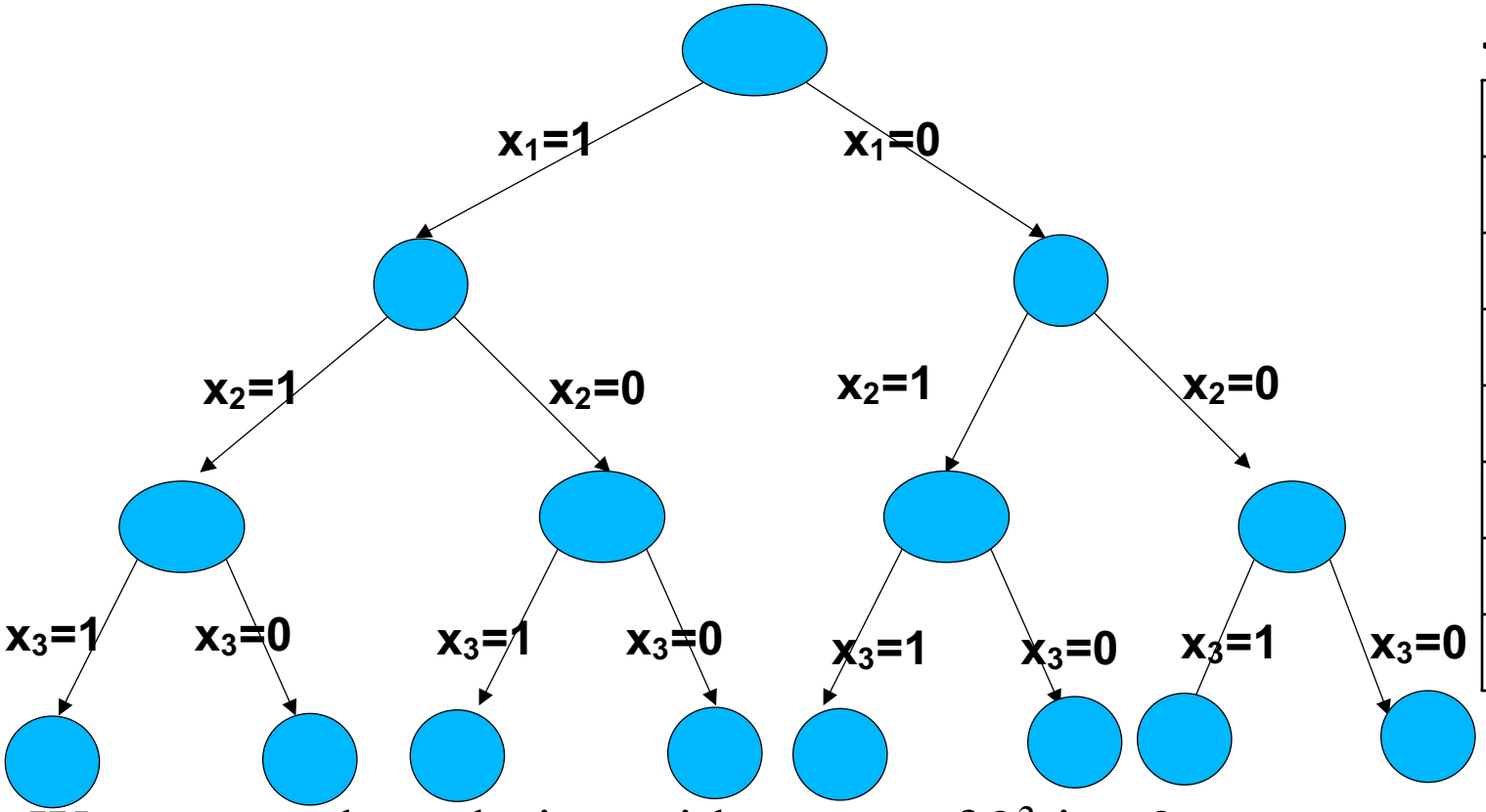
- P is the set of algorithms that takes polynomial time and deterministic algorithms.
- NP: is the set of algorithms that takes polynomial time but nondeterministic algorithms (**if Exponential time algorithms written in polynomial time**)
- P is a subset of NP,
- Some P algo are deterministic today,



but those are non-deterministic in past, example merge sort.

NP Hard and NP Complete

- If we are unable to solve any problem, then we should show that the similarity between so that in future, any one problem solved then other problems can solved in the same polynomial time.
- To relate these problems, we need a base problem.
- The base problem is CNF-satisfiability formula i.e propositional calculus formula using Boolean variables.
- Let Boolean variables $\{x_1, x_2, x_3\}$
- CNF=$\{x_1 V \dot{x}_2 V x_3\} \Lambda \{\dot{x}_1 V x_2 V \dot{x}_3\}$
- For which value of $\{x_1, x_2, x_3\}$ this becomes true.

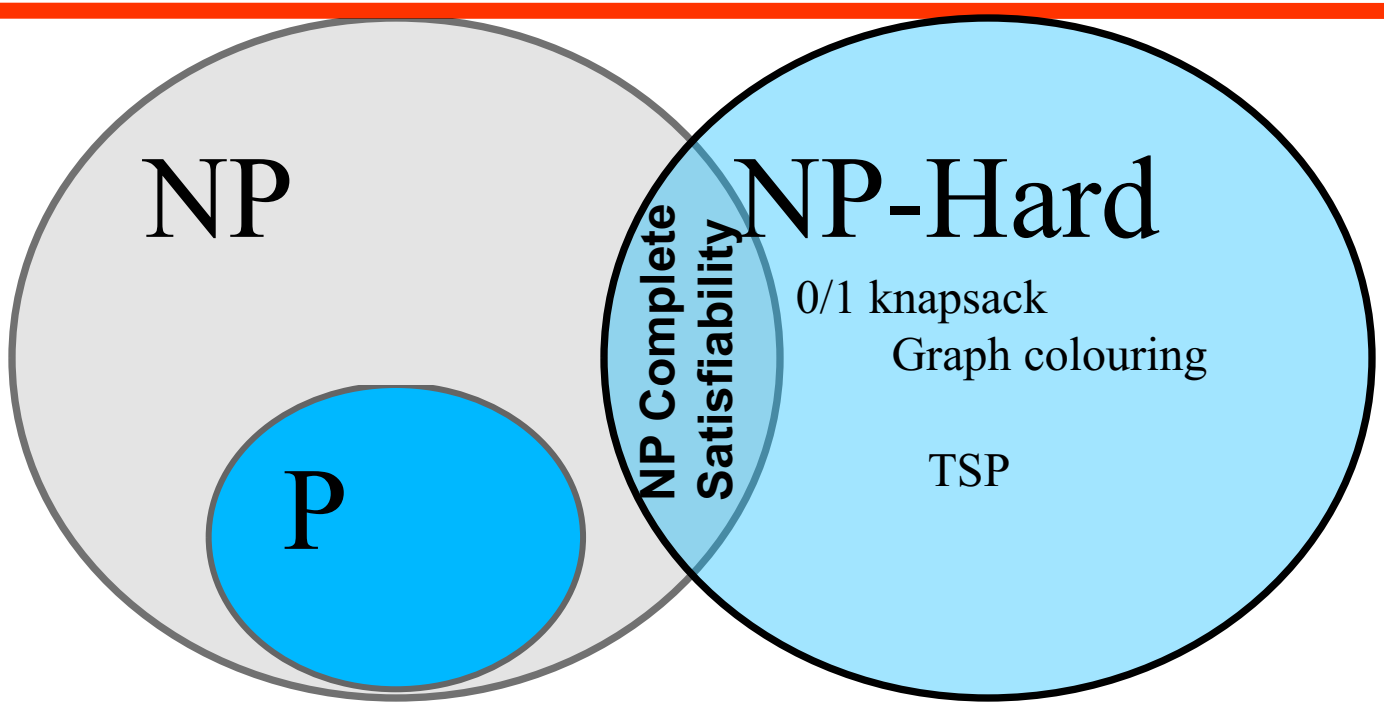


{x1,	x2,	x3}
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

We can get the solution with a cost of 2^3 i.e. 2^n

If CNF-satisfiability problem can be solved in polynomial time all these problems can be solved in polynomial time

- If any one of these can be solved in polynomial then Satisfiability problem can be solved in polynomial time using reduction.
- CNF Satisfiability➔ **reduced to** ➔ 0/1 Knapsack problem
- Satisfiability problem is a NP Hard, then 0/1 knapsack is also NP-Hard.



A problem X is NP-Complete if there is an NP problem Y, such that Y is reducible to X in polynomial time.

A problem is NP-complete if it is both in NP and NP-hard. The NP-complete problems represent the hardest problems in NP.

A Problem X is NP-Hard if there is an NP-Complete problem Y, such that Y is reducible to X in polynomial

- NP-Hard problems(say X) can be solved if and only if there is a NP-Complete problem(say Y) that can be reducible into X in polynomial time.
- NP-Complete problems can be solved by a non-deterministic Algorithm/Turing Machine in polynomial time.

Guest Lecture
on
Stoer-Wagner Algorithm for Maxflow / Min-cut

Prof. Ugo Fiore
Associate Professor, Department of Computer Science,
University of Salerno, Italy

Google Meeting Link: https://meet.google.com/yqo-osey-qwd